0042. Trapping Rain Water

ard | Category: Two Pointers | LeetCode: https://leetcode.com/problems/trapping

Problem

- Given bar heights, compute how much rain water is trapped between them.
- Example: [0,1,0,2,1,0,1,3,2,1,2,1] → 6.

Intuition

- Water above bar i = min(maxLeft(i), maxRight(i)) - h[i] (if positive).
- With two pointers, whichever side has the smaller running max is the bottleneck: its water level is already known, even without the full other side.

Brute Force

- For each i scan left and right for maxima: O(n^2).
- Precompute prefix-max and suffix-max arrays: O(n) time, O(n) space.

Optimal Approach

- L = 0, r = n-1, leftMax = rightMax = 0.
- If h[l] < h[r]: leftMax = max(leftMax, h[l]); water += leftMax - h[l]; l++.
- Else: rightMax = max(rightMax, h[r]); water += rightMax - h[r]; r--.

Worked Example

```
height = [4,2,0,3,2,5] (answer 9).

l=0(4) r=5(5) 4<5  leftMax=4 +0  l=1
l=1(2)        2<5  leftMax=4 +2  l=2
l=2(0)        0<5             +4  l=3
l=3(3)        3<5             +1  l=4
l=4(2)        2<5             +2  l=5 stop
total = 9
```

Java Solution

```java
public class TrappingRainWater {
    public int trap(int[] h) {
        int l = 0, r = h.length - 1, leftMax = 0, rightMax = 0, water = 0;
        while (l < r) {
            if (h[l] < h[r]) {
                leftMax = Math.max(leftMax, h[l]);
                water += leftMax - h[l++];
            } else {
                rightMax = Math.max(rightMax, h[r]);
                water += rightMax - h[r--];
            }
        }
        return water;
    }

    public int trapArrays(int[] h) {
        int n = h.length, water = 0;
        if (n == 0) return 0;
        int[] left = new int[n], right = new int[n];
        left[0] = h[0];
        for (int i = 1; i < n; i++) left[i] = Math.max(left[i - 1], h[i]);
        right[n - 1] = h[n - 1];
        for (int i = n - 2; i >= 0; i--) right[i] = Math.max(right[i + 1], h[i]);
        for (int i = 0; i < n; i++) water += Math.min(left[i], right[i]) - h[i];
        return water;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Update the max before adding water so the difference is never negative.
- Moving the side with the larger bar loses the guarantee.